

Strategic Fantasy Baseball Draft Optimization

Building a Scalable Decision-Support System for Front Offices

OR114-2 Final Project, Group 4

B13705051 周孟承 B13902103 鄭宇宏 B13303153 詹舒宇 B11705039 李盈盈

Spring 2026

Abstract

For a modern Major League Baseball (MLB) franchise, the draft is the most cost-effective avenue for building a championship roster. As internal scouting and player evaluation models converge across the league, the competitive edge shifts from merely identifying talent to executing a mathematically superior drafting strategy. In this report, we model the draft as an operations research problem. Because the pool of draft-eligible players is massive, traditional Integer Linear Programming (ILP) fails to scale. We propose a highly efficient heuristic, *Opportunity Cost Greedy (OCG)*, which estimates the delay-cost of waiting for future turns. Tested across various synthetic market stressors and real-world 2026 projections, OCG delivers near-optimal rosters in a fraction of a second, effectively bridging the gap between exact optimization and massive-scale practicality.

1 Introduction: The Front Office Perspective

For any MLB front office, constructing a roster before the season begins is the ultimate optimization puzzle. The most efficient and critical way to acquire talent is through the draft. Every team possesses internal scouting models and proprietary projections (e.g., WAR, fantasy points) to evaluate players. However, in the modern data-driven era, these evaluations often converge. When all general managers have access to similar player rankings, the true competitive advantage is no longer just *who* you evaluate as good, but *when* and *how* you draft them relative to your opponents and your roster constraints.

While the actual MLB draft uses a fixed order based on reverse standings, we abstract our environment into a **Snake Draft**. This format serves as a perfect proxy to simulate a highly competitive, fair, and strategically demanding environment. In a snake draft, a manager owns a fixed sequence of picks, and the player pool is continually depleted by opponents. The practical decision is not simply selecting the player with the highest isolated score (the naive “Best Player Available” approach). Instead, a manager must answer a harder question: *Which player must I draft now because market timing and positional scarcity will make equivalent options disappear by my next turn?*

This decision is inherently interdependent. A high-projection player might be suboptimal if he occupies a position where supply is abundant, whereas a lower-projection player might be highly valuable if he fills a scarce roster slot. To solve this, we translate the draft into a mathematical model.

Initially, an Average Draft Position (ADP)-aware **Integer Linear Program (ILP)** is formulated. While ILP provides a flawless mathematical optimum, it encounters a severe roadblock in reality. Because baseball players do not need to actively register for the draft (anyone meeting age/school

requirements is eligible), the actual draft pool scales to tens of thousands of players. When the problem scale rises, the ILP algorithm exceeds reasonable time limits or crashes completely due to exponential complexity.

To solve this, we introduce a self-designed algorithm: **Opportunity Cost Greedy (OCG)**. This heuristic explicitly calculates the mathematical cost of waiting by peeking into the future availability of players. The overarching narrative of this report is to prove to the front office that OCG can achieve incredibly tight optimality gaps (retaining maximum roster value) while executing in mere seconds, even when the player pool scales to massive extremes.

2 Problem Description

We consider a snake draft environment where a manager operates a predetermined set of picks. At each owned pick, the manager must choose exactly one available player from the remaining pool. Every player in the pool possesses three attributes:

1. **Projected Value:** The expected statistical output (points) for the season.
2. **Eligible Positions:** The real-life positions the player can occupy.
3. **Average Draft Position (ADP):** A market consensus of when the player is expected to be drafted by opponents.

The objective is to maximize the sum of projected points for the starting roster. The drafted players must perfectly fulfill strict positional requirements. The base roster structure used in our study demands 16 active slots, as detailed in Table 1. (Note: The Utility (**Util**) slot is hitter-only, meaning pitchers cannot be assigned there).

Table 1: Base fantasy baseball roster requirements.

Position	C	1B	2B	3B	SS	OF	Util	SP	RP
Required Slots	1	1	1	1	1	3	1	5	2

A player can only be drafted if he is still available in the market. We treat ADP as a proxy for market depletion. If the manager’s current draft pick occurs much later than a player’s ADP, that player is considered successfully drafted by an opponent and is unavailable. By integrating ADP market constraints and positional flexibility into the sequence of decisions, draft planning is transformed from a subjective ranking exercise into a strict operations research problem.

3 Model Formulation

We first formulate the exact ADP-aware Integer Linear Program (ILP) to serve as our absolute benchmark for roster quality.

Let I be the set of players, P the set of roster positions, and K the set of owned draft picks. Each player $i \in I$ has projected points v_i , market expectation adp_i , and eligible positions $E_i \subseteq P$. Each position $p \in P$ requires r_p players. Let S_k be the overall draft number for the manager’s k -th pick.

Decision Variables:

- $y_i \in \{0, 1\}$: 1 if player i is drafted, 0 otherwise.
- $x_{ip} \in \{0, 1\}$: 1 if player i is assigned to position p , 0 otherwise.

- $z_{ik} \in \{0, 1\}$: 1 if player i is selected precisely at owned pick k , 0 otherwise.

Objective Function: Maximize total roster points.

$$\max \sum_{i \in I} v_i y_i$$

Constraints:

$$\sum_{i \in I} z_{ik} = 1, \quad \forall k \in K \quad (1)$$

$$\sum_{k \in K} z_{ik} = y_i, \quad \forall i \in I \quad (2)$$

$$\sum_{p \in P} x_{ip} = y_i, \quad \forall i \in I \quad (3)$$

$$x_{ip} \leq \mathbf{1}(p \in E_i), \quad \forall i \in I, p \in P \quad (4)$$

$$\sum_{i \in I} x_{ip} = r_p, \quad \forall p \in P \quad (5)$$

$$z_{ik} = 0 \quad \text{if } S_k > \text{adp}_i + \delta, \quad \forall i \in I, k \in K \quad (6)$$

$$y_i, x_{ip}, z_{ik} \in \{0, 1\} \quad \forall i, p, k \quad (7)$$

Constraint (1) enforces exactly one selection per pick. (2) and (3) ensure logical consistency between picking, drafting, and assigning. (4) restricts assignments to legal positions. (5) dictates the roster shape. Constraint (6) is the vital **Market Availability Constraint**: a player cannot be drafted if the current pick S_k is later than his expected depletion point (ADP plus a tolerance buffer δ).

4 Algorithms

While the ILP provides mathematical perfection, solving it is NP-Hard. For massive player pools, the branch-and-bound tree explodes. Therefore, we developed two heuristic algorithms using Priority Queues (Heaps) to achieve extreme computational efficiency.

4.1 Direct Greedy (Baseline)

The Direct Greedy (DG) algorithm serves as our baseline. It represents the standard “fill the biggest need with the best available player” mentality.

At each pick, DG scans all open positions, identifies the position with the highest scarcity (Remaining Need / Available Players), and selects the player with the highest projected points for that position.

Algorithm 1 Direct Greedy (DG)

```
1: Initialize Max-Heaps for each position  $p \in P$  based on  $v_i$ .
2: Sort all players by  $(\text{adp}_i + \delta)$  to create an expiration_order.
3: for each owned pick  $k \in K$  do
4:   Pop expired players from heaps where  $\text{adp}_i + \delta < S_k$ 
5:    $\text{best\_position} \leftarrow \text{null}$ ,  $\text{max\_scarcity} \leftarrow -\infty$ 
6:   for each position  $p$  with  $r_p > 0$  do
7:     Clean heap (remove drafted/expired players lazily)
8:      $\text{scarcity} \leftarrow \frac{\text{Remaining Need for } p}{\text{Active Eligible Players for } p}$ 
9:     if  $\text{scarcity} > \text{max\_scarcity}$  (tie-break by top player's  $v_i$ ) then
10:       $\text{best\_position} \leftarrow p$ 
11:       $\text{max\_scarcity} \leftarrow \text{scarcity}$ 
12:     end if
13:   end for
14:   Draft top player from  $\text{best\_position}$  heap and update remaining needs.
15: end for
```

4.2 Opportunity Cost Greedy (Proposed Method)

Opportunity Cost Greedy (OCG) is our flagship algorithm. Instead of only looking at the *current* turn, OCG looks ahead to the *next* owned pick. It calculates the **Opportunity Cost** of waiting for a position by subtracting the points of the best player available *next round* from the best player available *now*.

Algorithm 2 Opportunity Cost Greedy (OCG)

```
1: Initialize two sets of Max-Heaps for each position  $p$ : current_heaps and future_heaps.
2: for each owned pick  $k \in K$  (let next pick be  $k_{\text{next}}$ ) do
3:   Expire players in current_heaps based on  $S_k$ 
4:   Expire players in future_heaps based on  $S_{k_{\text{next}}}$ 
5:    $\text{best\_choice} \leftarrow \text{null}$ ,  $\text{max\_priority} \leftarrow (-\infty, -\infty, -\infty)$ 
6:   for each position  $p$  with  $r_p > 0$  do
7:      $v_{\text{cur}} \leftarrow$  points of top player in current_heaps[ $p$ ]
8:      $v_{\text{fut}} \leftarrow$  points of top player in future_heaps[ $p$ ]
9:      $\text{score} \leftarrow v_{\text{cur}} - v_{\text{fut}}$  ▷ Delay Cost Calculation
10:     $\text{forced} \leftarrow \text{True}$  if (Active Players  $\leq$  Need)
11:     $\text{priority} \leftarrow (\text{forced}, \text{score}, v_{\text{cur}})$ 
12:    if  $\text{priority} > \text{max\_priority}$  then
13:       $\text{max\_priority} \leftarrow \text{priority}$ 
14:       $\text{best\_choice} \leftarrow (\text{Top Player}, p)$ 
15:    end if
16:  end for
17:  Draft player from  $\text{best\_choice}$  and update remaining needs.
18: end for
```

4.3 Time Complexity Analysis

Let N be the total number of players in the draft pool, R be the number of roster slots (total picks per manager), and P be the number of distinct positions.

- **Initialization:** Building the initial max-heaps and sorting the expiration list takes $O(N \log N)$.
- **Drafting Loop:** For each of the R picks, the algorithm scans P positions. Checking the top of a heap is $O(1)$.

- **Heap Cleaning (Lazy Deletion):** Expiring or removing drafted players is done by popping invalid elements at the top of the heap. Across the *entire draft process*, a player is popped from a position heap at most once. Therefore, the total time spent cleaning heaps is bounded by $O(N \log N)$.

Total Time Complexity for both DG and OCG is $O(N \log N + R \cdot P)$. Because R and P are tiny constants relative to N , the algorithm runs in near-linear time relative to sorting the player pool. This mathematically guarantees extreme scalability compared to the exponential $O(2^V)$ worst-case nature of ILP.

5 Data Collection and Generation

To rigorously test our algorithms and simulate the vast ecosystem of MLB draft-eligible players, we developed a highly customizable **Synthetic Data Generator**. While real data (2026 Yahoo/FanGraphs projections) is used for final validation, synthetic data is indispensable. It allows us to mathematically isolate specific operational variables (Factors)—such as talent distribution, positional versatility, and market efficiency—and stress-test the algorithms under controlled extreme conditions.

The total number of generated players is defined dynamically based on the league size (T), base roster slots ($R = 16$), roster scale multiplier (S), and player-demand ratio (D):

$$N_{players} = S \times 16 \times T \times D$$

Our experimental framework is divided into 6 distinct Scenarios (N1 to N6). We generate synthetic profiles encompassing three primary dimensions:

5.1 Factor 1: Projected Points Distribution (v_i)

We designed three points distributions to simulate different talent pools, clipping all generated values to a realistic range of $[10.0, 800.0]$:

- **Normal (Baseline):** Values follow a normal distribution $v_i \sim \mathcal{N}(400, 120^2)$. This represents an average draft class where the majority of players are solid, mid-tier contributors.
- **Uniform:** Values are uniformly distributed $v_i \sim \mathcal{U}(10, 800)$. This flattens the talent curve, offering many viable replacement options and minimizing scarcity.
- **High-Low (Star-Heavy):** Simulates a highly skewed draft class. Only 10% of players are classified as elites, with values drawn from $\mathcal{U}(500, 800)$. The remaining 90% fall into $\mathcal{U}(10, 500)$. In this scenario, missing out on top-tier talent is severely punished in the objective function.

5.2 Factor 2: Position Eligibility Distribution (E_i)

We control how versatile the player pool is using four distinct generation logics:

- **Roster-Ratio (Baseline):** Players are assigned exactly one position, but the probability of generating a specific position aligns precisely with roster demand (e.g., more OF and SP players are generated).
- **Uniform-by-Type:** Randomly designates a player as a hitter or pitcher, then uniformly assigns a position pattern (including dual-eligibility like 1B;3B or SP;RP).

- **Point-Flexible:** The most forgiving environment. The generator ranks all players by points. Players in the top 20% (elite) have an 80% chance of gaining multi-position eligibility (up to 3 positions). This mimics real life where superstars often possess high versatility.
- **Single-Position:** Every player possesses exactly one eligible position. This eliminates all flexibility, representing the ultimate test of scarcity and making early greedy mistakes almost impossible to repair.

5.3 Factor 3: Market Volatility and ADP Noise (σ_{adp})

To generate ADP, we first sort all players by their true projected points in descending order to establish a “True Rank”. We then simulate market noise by adding a Gaussian error term:

$$\text{ADP}_i = \text{TrueRank}_i + \mathcal{N}(0, \sigma_{\text{adp}}^2)$$

- $\sigma_{\text{adp}} = 0$ represents a perfectly efficient market where opponents draft strictly by objective value.
- Higher σ_{adp} introduces market chaos. Opponents may make unpredictable “reaches” (drafting a player too early) or leave bargains on the board. This tests how robust our algorithms are when future availability is highly uncertain.

Table 2 outlines the configuration for the 6 experiment sets, manipulating these factors systematically.

Table 2: Synthetic Experiment Design. Bold text highlights the varied factor in each scenario block.

Scenario	Goal	Points Dist.	Positions Dist.	Scale & Demand	ADP Noise (σ, δ)
N1	Baseline	normal	roster-ratio	S=1, D=3	(30, 0)
N2	Points Effect	varied (3)	roster-ratio	S=1, D=3	(30, 0)
N3	Positions Effect	normal	varied (4)	S=1, D=3	(30, 0)
N5	Market Volatility	normal	roster-ratio	S=1, D=3	varied (5), $\pm\delta$
N4	Scaling Growth	normal	roster-ratio	S=1-3, D=1-10	(30, 0)
N6	Stress Test	high-low	single-position	Massive Scale	high noise

6 Performance Evaluation

Algorithms are evaluated primarily using the **Optimal Gap Ratio** relative to the ADP-aware ILP benchmark:

$$\text{Gap Ratio} = \frac{Z_{\text{ILP}} - Z_{\text{Heuristic}}}{Z_{\text{ILP}}} \times 100\%$$

where Z is the total projected points. A smaller gap indicates superior performance.

6.1 N1 to N3: Base, Points, and Positional Flexibility

Table 3 summarizes the results for the fundamental scenarios. In the **N1 Baseline**, our proposed Opportunity Cost Greedy (OCG) achieves a tight gap of 1.92%, reducing the error of the naive Direct Greedy baseline (4.79%) by more than half.

The true strategic value of OCG is revealed in **N2 (Points Dist.)**. When the draft class shifts to *high-low* (star-heavy), Direct Greedy’s gap balloons to 8.39%. Direct Greedy myopically drafts for immediate positional needs, allowing elite superstars to slip away to opponents. OCG explicitly

calculates the delay-cost, realizes it cannot afford to pass on elites (the drop-off in value for the next round is massive), and secures a much superior 2.87% gap.

In **N3 (Positions Dist.)**, *point-flexible* is the most forgiving environment (gap drops to 1.34%) because top players can seamlessly plug holes in the roster. Conversely, *single-position* creates a rigid, unforgiving draft. Even when mistakes are impossible to patch with utility players, OCG dominates Direct Greedy (0.94% vs 5.11%), proving that looking one pick ahead prevents catastrophic positional deficits later in the draft.

Table 3: Results for Scenarios N1, N2, and N3. Values represent optimal gap ratio (% $\pm$ std).

Scenario	Level	Direct Greedy	Opportunity Cost Greedy
N1: Baseline	normal	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%
N2: Points Dist.	uniform	3.24% $\pm$ 0.77%	1.44% $\pm$ 0.59%
	high-low	8.39% $\pm$ 2.40%	2.87% $\pm$ 1.32%
N3: Pos Dist.	uniform-by-type	4.89% $\pm$ 0.62%	1.07% $\pm$ 0.43%
	point-flexible	5.36% $\pm$ 1.66%	1.34% $\pm$ 0.68%
	single-position	5.11% $\pm$ 1.48%	0.94% $\pm$ 0.65%

6.2 N5: Market Volatility (ADP Uncertainty)

Table 4 demonstrates performance under chaotic draft markets. When the market is perfectly efficient ($\sigma_{\text{adp}} = 0$), predicting opponent behavior is trivial, and both algorithms perform exceptionally well.

As noise increases to $\sigma_{\text{adp}} = 60$, the draft becomes unpredictable. Direct Greedy struggles significantly (gap rises to 5.73%), whereas OCG remains highly robust (2.43%). Because OCG bases its priority on the *difference* (Opportunity Cost) between current and future availability, it inherently hedges against market volatility far better than relying on absolute value rankings.

Table 4: N5 ADP Uncertainty results. Values are optimal gap ratio (% $\pm$ std).

Scenario	Level	Direct Greedy	Opportunity Cost Greedy
N5: Market Noise	$\sigma_{\text{adp}} = 0$ (Perfect)	1.72% $\pm$ 0.70%	0.42% $\pm$ 0.17%
	$\sigma_{\text{adp}} = 10$ (Mild)	3.34% $\pm$ 0.91%	0.99% $\pm$ 0.45%
	$\sigma_{\text{adp}} = 30$ (Moderate)	5.03% $\pm$ 1.28%	1.80% $\pm$ 0.75%
	$\sigma_{\text{adp}} = 60$ (High)	5.73% $\pm$ 1.35%	2.43% $\pm$ 1.38%

6.3 Scalability and Stress Testing (N4 & N6)

This section presents the most critical argument for our front office pitch: **Scalability**. While the ILP finds perfect rosters in smaller instances, what happens when the scouting department dumps a universe of 180,000 eligible players into the database?

Table 5 highlights N4 scaling under tight vs. loose supply. When supply is brutally tight (Demand Ratio = 1, meaning almost every player must be drafted), Direct Greedy collapses (8.13% gap) as it blindly paints itself into a corner. OCG, aware of the impending scarcity, navigates the tight market with only a 3.02% gap.

Table 5: N4 Scaling Results (Selected). Values are optimal gap ratio ($\% \pm \text{std}$).

Environment	Direct Greedy	Opportunity Cost Greedy
Scale 1, Tight Supply (D=1)	8.13% $\pm$ 1.64%	3.02% $\pm$ 0.95%
Scale 1, High Supply (D=10)	3.39% $\pm$ 0.96%	1.23% $\pm$ 0.38%
Scale 3, Tight Supply (D=1)	4.82% $\pm$ 0.40%	2.69% $\pm$ 0.55%

Table 6 showcases the **N6 Large-Scale Stress Test**. As we scale the roster size to 256 and the player pool to 184,320, the approximate variable count for the ILP reaches nearly 50 million. The Branch-and-Bound tree of the ILP explores an exponentially growing search space. As a result, the ILP runtime explodes from 4 seconds to 276 seconds, eventually triggering a `TIMEOUT_ERROR` (exceeding 1800 seconds) in the largest target.

In stark contrast, confirming our $O(N \log N)$ complexity analysis, **Opportunity Cost Greedy processes the massive 50-million variable equivalent environment in just 23.286 seconds**, utilizing hyper-efficient lazy deletion on its priority queues.

Table 6: N6 Stress Test. The exponential collapse of ILP versus the linear stability of OCG.

Instance Level	Approx. ILP Vars	Players	DG Time(s)	OCG Time(s)	ILP Time(s) / Status
stress_small	334,080	5,760	0.790	0.852	4.235 / OPTIMAL
stress_medium	710,400	9,600	0.951	1.144	12.230 / OPTIMAL
stress_large	2,289,600	21,600	1.981	2.344	54.267 / OPTIMAL
stress_xlarge	10,880,000	64,000	6.172	10.360	276.895 / OPTIMAL
timeout_target	49,029,120	184,320	17.760	23.286	1869.805 / TIMEOUT

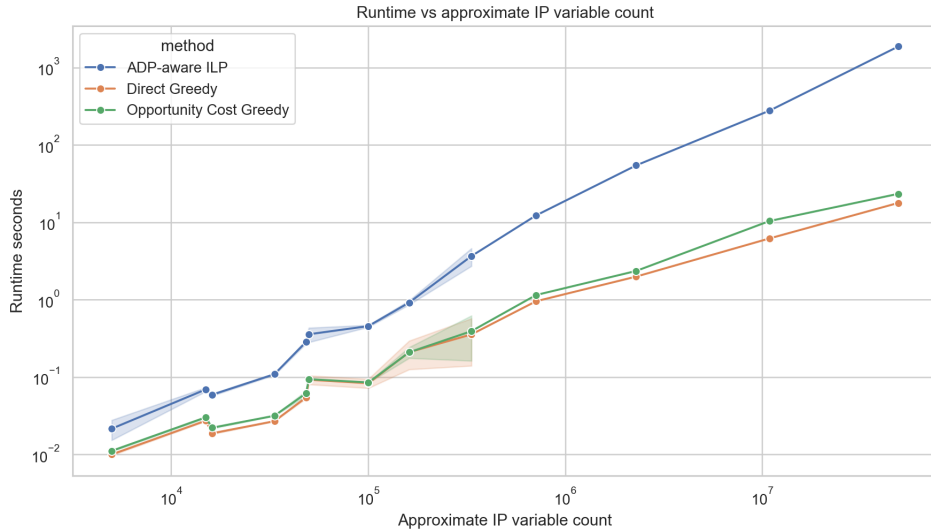


Figure 1: Runtime comparison for N6. ILP scales exponentially, while heuristics scale linearly.

6.4 Real-Data Validation

To ensure practical viability for the upcoming season, we ran the algorithms on 2026 Yahoo and FanGraphs projections. As seen in Table 7, OCG successfully trims the point loss caused by naive greedy drafting, confirming that synthetic insights hold true in real MLB data structures.

Table 7: Real-data validation (Gap measured in absolute projected points lost vs ILP).

Dataset	Method	Mean Obj. Points	Mean Gap	Max Gap
Yahoo 2026	ADP-aware ILP	7917.209	0.000	0.000
	Opportunity Cost Greedy	7767.732	149.477	255.700
	Direct Greedy	7662.866	254.343	373.000
FanGraphs 2026	ADP-aware ILP	14642.927	0.000	0.000
	Opportunity Cost Greedy	14320.135	322.792	508.000
	Direct Greedy	14097.337	545.590	644.500

7 Conclusions and Business Recommendations

For an MLB front office, a successful draft dictates the trajectory of the franchise. While internal scouting evaluations have largely converged across the league, draft execution remains a highly exploitable inefficiency.

This project successfully translated the chaotic, interdependent environment of a fantasy snake draft into a rigorous Operations Research model. We demonstrated that relying on naive “best player available” mentalities (Direct Greedy) bleeds significant roster value, especially when the talent pool is top-heavy or positional flexibility is low.

While the exact ADP-aware ILP provides flawless rosters, the limitless nature of the baseball player pool renders exact optimization operationally dead-on-arrival due to exponential runtime scaling. Our proposed **Opportunity Cost Greedy (OCG)** algorithm brilliantly bridges this gap. By calculating the delay-cost of waiting just one pick into the future, OCG captures the strategic essence of the ILP. It consistently minimizes the optimality gap under severe market stress and processes 50-million variable equivalent draft instances in less than 25 seconds.

Limitations & Future Extensions: ADP is only a proxy for opponent behavior; human managers exhibit biases (e.g., hometown favorites) that ADP misses. Furthermore, point projections carry standard deviations (injury risk, slump risk) not currently modeled. Future iterations of this tool could incorporate probabilistic opponent modeling and Monte Carlo simulations on projection variance, upgrading OCG from a deterministic solver to a fully probabilistic draft-day AI.

Ultimately, this report proves to the front office that computational draft optimization is not just a theoretical exercise, but a scalable, real-time weapon capable of maximizing roster value on draft day.